Enabling Data Privacy in Blockchain

Senthilnathan Natarajan, Manish Sethi, Dave Enyeart, Yacov Manevich, Artem Barger
IBM Research

ABSTRACT

Blockchain technology provides a decentralized and immutable ledger, but its inherent transparency poses a significant barrier to enterprise adoption in domains requiring data privacy. Existing solutions like channels provide data isolation but are coarse-grained, requiring a separate ledger and consensus instance per channel. In this paper, we introduce *private data collections*, a mechanism for managing private data on permissioned blockchains. Our approach decouples private data storage from the replicated ledger: private data is shared only among authorized peers, while cryptographic hashes are recorded on the ledger for tamper-evidence and serializability verification. The key theoretical contribution is **blind validation**: we prove that on-chain hashes form a *conflict-complete proxy* for private state—OCC-based serializability checks on hashed state detect exactly the same conflicts as checks on the underlying private data, producing neither false negatives nor false positives. This enables the entire network to verify the correctness and serializability of private transactions without ever accessing the private data itself. We design a two-phase gossip-based protocol for proactive dissemination and reactive retrieval of private data, and we formalize the collection abstraction with seven design requirements. Our solution is designed for any blockchain platform that follows the simulate-order-validate-commit replication protocol. We implement it in Hyperledger Fabric and demonstrate that private data collections add only 3–14% overhead to transaction throughput while providing fine-grained data privacy. The mechanism has been in production since Hyperledger Fabric v1.2 and is deployed by organizations including Walmart, MediLedger, Honeywell, Oracle and GSBN.

PVLDB Reference Format:
Senthilnathan Natarajan, Manish Sethi, Dave Enyeart, Yacov Manevich, Artem Barger. Enabling Data Privacy in Blockchain. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Enterprise blockchain networks are redefining organizational collaboration by establishing trust without a central intermediary. In sectors ranging from food safety [22, 42] and pharmaceuticals [38, 40] to global trade [25] and mining [24], these systems utilize a permissioned, decentralized ledger to provide a *single source of truth* for high-stakes workflows—reducing audit times from days to seconds and ensuring compliance with regulatory frameworks such as the U.S. Drug Supply Chain Security Act [39] and the EU General Food Law [14]. The value of these deployments stems from the blockchain's core properties: a transparent, immutable, and tamper-evident audit trail shared among mutually distrusting parties. However, a fundamental tension exists between this transparency and the privacy requirements of enterprise data. Sensitive information—such as negotiated pricing, HIPAA-protected medical records [20], or GDPR-governed identities [15]—cannot be fully replicated across a ledger where it is exposed to every participant, including competitors. Yet keeping such data entirely off-chain forfeits the dispute-resolution and regulatory-compliance guarantees that make blockchain valuable (§2). This creates a *privacy-auditability* deadlock: participants must prove the validity of private transactions to the network without revealing the underlying sensitive data.

Resolving this deadlock requires two properties that are individually straightforward but difficult to combine: *fine-grained privacy*, where private data remains visible only to authorized parties, and *verifiable auditability*, where on-chain cryptographic evidence allows any participant to verify integrity and trace provenance. In practice, achieving both on a shared ledger imposes additional constraints. A single business transaction frequently updates both public and private state, so the two must commit or abort atomically; a partial commit leaves the on-chain evidence inconsistent with reality. Furthermore, because privacy rules are deeply tied to business logic (who may read a price depends on the contract governing the trade), enforcement must be embedded in the same general-purpose smart contracts—a separate mechanism that restricts expressiveness or imposes prohibitive cost defeats the purpose. Finally, a practical deployment constraint: the solution must be *hardware agnostic*, operating on commodity infrastructure without dependence on specialized processors whose availability, cost, or side-channel vulnerabilities narrow the set of participants that can join the network. Table 1 evaluates existing approaches against these requirements.

Table 1: Comparison of Privacy Mechanisms

Mechanism	Fine-Grained Privacy	Cross-State Atomicity	Smart Contract Access Control	Hardware Agnostic
Channels	×	×	✓	✓
ZKPs	✓	✓	Limited[1]	✓
TEEs (e.g., SGX)	✓	✓	✓	×
Off-chain DB	✓	×	×	✓
Pvt. Collections	✓	✓	✓	✓

As the table shows, no existing approach satisfies all four requirements. *Channels* [1] lack fine-grained privacy: each channel is an entirely separate ledger, so every bilateral data-sharing relationship requires its own channel with its own consensus instance, scaling poorly. They also lack cross-state atomicity, since atomic transactions spanning data on separate channels require fragile cross-ledger coordination. *Zero-knowledge proofs* [2, 26] provide fine-grained privacy and atomicity but restrict smart contract

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Yacov Manevich is now at Ava Labs. Artem Barger is now at Modelyo.

expressiveness to arithmetic circuits or constraint systems, with per-proof overhead that remains prohibitive for high-throughput enterprise workloads. *Trusted Execution Environments* [9, 37] satisfy the privacy, atomicity, and smart contract requirements but are not hardware agnostic—they depend on specialized processors with well-documented side-channel vulnerabilities. Off-chain databases offer fine-grained privacy without hardware dependencies but sacrifice cross-state atomicity and place private data outside smart contract governance (§2).

This paper formalizes the principle of *decoupled storage with on-chain cryptographic anchors* through the *Private Data Collections* (PDCs) mechanism. The core of our approach is *blind validation*: by decoupling the storage of private plaintext from its cryptographic commitment, the entire network can verify the serializability and integrity of transactions without accessing the underlying data. Authorized peers process private data in plaintext and concurrently record its hash on the replicated ledger; at commit time, any node can validate state conflicts by comparing hash versions alone. This architecture preserves transactional atomicity across public and private states within a single shared ledger—eliminating the management overhead of channels, the high computational cost of ZKPs, and the hardware dependencies of TEEs. However, excluding private data from the consensus-driven block delivery introduces a critical data availability challenge: ensuring that all authorized peers obtain the required private state before block commitment. We address this with a robust dual-path dissemination protocol: a proactive gossip-based mechanism that propagates data during transaction simulation, complemented by a reactive pull-based recovery for nodes recovering from network partitions or dynamic membership changes.

The main contributions of this paper are as follows:

- We formalize the private data collection abstraction, identify seven design requirements (atomicity, hash-data integrity, smart contract access control, authorized execution, dynamic membership, data lifecycle management, and data availability), and analyze replication protocols to determine their suitability for private data (§3).
- We design a dual-layered storage architecture (§4) built on a *blind validation* mechanism, and formally prove that on-chain hashes serve as a conflict-complete proxy for private state (Theorem 1), enabling network-wide serializability verification without exposing private data.
- We propose a two-phase gossip-based protocol (§4) that combines proactive dissemination with reactive retrieval, handles dynamic membership changes, and tolerates network partitions.
- Our design is applicable to any blockchain platform that implements the simulate-order-validate-commit replication protocol. We implement PDCs in Hyperledger Fabric and report on production deployment experience since v1.2 (with continued enhancements through v2.5); organizations including Walmart [22, 42], MineHub [24], GSBN [25], MediLedger [38, 40], Honeywell [21], and Oracle [33] deploy it today. Our evaluation across four industry workloads demonstrates that private data transactions incur 3–14% overhead relative to public-only transactions.

2 MOTIVATION FOR DATA PRIVACY

Real-world blockchain deployments span diverse domains—supply chains, healthcare, trade finance—where organizations must share data with relevant parties for coordination while simultaneously protecting it from unauthorized participants and maintaining auditability for the entire network. The dual requirement—*share with a subset, auditable by all*—is the defining challenge. We illustrate it through food-supply-chain traceability, a domain where the tension between privacy and auditability is especially acute.

Food Traceability. In 2006, a major *E. coli* outbreak linked to bagged spinach infected 199 people across 26 U.S. states [32]. Regulators took two weeks to trace the contamination source, during which significant quantities of safe food were wasted [18]. A blockchain-based traceability system addresses this by providing a tamper-evident, shared record—Walmart demonstrated that blockchain could reduce backtrace time from seven days to 2.2 seconds [22, 42]. However, not all data in a food supply chain has the same visibility requirements. Invoices, pricing terms, quantities, and discount schedules are exchanged strictly between a buyer and a seller—they must remain private from other participants in the network, including competitors transacting with the same partners.

Why private data must still be on-chain. Yet these records cannot simply be kept off-chain. When a buyer and seller later dispute the agreed price or delivery terms, the blockchain's immutable record serves as an authoritative witness that neither party can retroactively alter. Beyond conflict resolution, when an outbreak triggers an FDA investigation and companies must disclose shipment records, the regulator—and indeed any consortium member—must be able to verify that the disclosed records are genuine and unaltered, even if they never held the private data. On-chain cryptographic evidence makes this possible: the immutable commitment acts as a cryptographic anchor, allowing a regulator to verify a disclosed record against the ledger without the network ever having stored the plaintext—retroactive fabrication becomes impossible.

Atomicity across public and private state. Updating a public shipment record and its private invoice must succeed or fail together—if one commits without the other, the on-chain hash no longer reflects reality and the audit trail becomes untrustworthy. Food standards authorities [13, 16, 17] mandate detailed record-keeping, yet much of it is commercially sensitive. Organizations need fine-grained control over what is shared with whom, without sacrificing the auditability that makes the traceability system trustworthy. Relegating private data to an external database forces organizations to manage a *shadow* coordination layer—replicating data to authorized parties, enforcing access revocation, and maintaining consistency with the on-chain state—all independently of the blockchain, duplicating operational overhead that the blockchain node already subsumes. Integrating privacy into the ledger's native state machine is the only way to guarantee atomicity while eliminating this redundant infrastructure.

Privacy as a Participation Prerequisite. Beyond regulatory compliance, transparency creates an economic barrier to participation. Consortium members are often competitors, and blockchain's value increases with participation—network effects are central to its appeal—yet transparency gives organizations a rational incentive

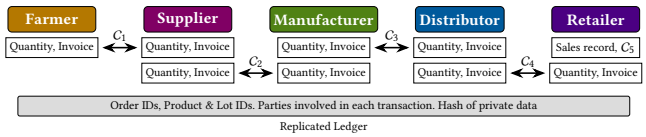

Figure 1: Private data is visible only to authorized parties; on-chain hashes provide proof of existence and traceability.

to withhold private data or avoid joining altogether. The audit guarantee is what makes organizations willing to join despite privacy concerns: each member trusts that on-chain evidence protects the integrity of every transaction, even transactions involving private data they cannot see. Privacy is therefore not merely a compliance requirement; it is a prerequisite for consortium formation, and auditability is the trust mechanism that enables it. These requirements motivate the *private data collection* abstraction formalized in the next section.

3 PRIVATE DATA COLLECTIONS: CONCEPT AND DESIGN REQUIREMENTS

To resolve the privacy-auditability tension identified in §2, we integrate private data directly into the blockchain node, giving rise to the *private data collection* mechanism.

3.1 Key Idea

The mechanism rests on two complementary pillars: (1) *decoupled storage*, where private data is shared and stored only on peers owned by member organizations of a *collection* with defined membership—ensuring *privacy*; and (2) *on-chain evidence*, where the cryptographic hash of private data is recorded on the replicated ledger—ensuring *auditability*. A single transaction may modify both public and private data simultaneously, and the two must commit atomically; a partial commit renders the on-chain evidence meaningless.

Formally, a private data collection is defined as a tuple $C = (id, \mathcal{M}, \tau)$ with the following parameters:

- id: unique collection identifier
- $\mathcal{M}$: membership policy specifying the set of authorized organizations
- τ: block-to-live (BTL) retention—the number of blocks after which private data is purged, enabling compliance with data-retention regulations; $\tau = 0$ means indefinite retention

A smart contract may define multiple collections, each with independent membership policies. Additional implementation-specific parameters—such as authorization policies governing write access and dissemination thresholds—are discussed in §4.

Figure 1 illustrates the mechanism in the food traceability setting. Five collections (C_1–C_5) are defined along participant relationships: C_1–C_4 each span two adjacent parties in the supply chain, while C_5 is private to the Retailer alone. Public data—order IDs, product lots—lands on the replicated ledger alongside a hash for each private data element; the actual quantities, invoices, and pricing terms are stored only in the relevant collection. Competitors can verify product provenance without ever seeing each other's contracts, and any participant can confirm a disclosed record's authenticity by checking it against the pre-existing on-chain hash. Because each hash is committed as part of a block, the ledger provides a tamper-evident timeline that all participants—even those that never held the private data—can trust.

3.2 Design Requirements

Table 1 captured a basic set of differentiators among privacy mechanisms. The use case in §2 and the architectural constraints of our key idea give rise to a more comprehensive set of seven design requirements, across three categories.

(1) Correctness. Integrating private data into a blockchain must not weaken the transactional guarantees. As the food traceability scenario illustrates, updating a public record and its private counterpart must be atomic—a partial commit leaves the ledger inconsistent and the on-chain hash meaningless. Beyond atomicity, the hash recorded on the ledger must faithfully reflect the off-chain data at all times; otherwise, the auditability guarantee evaporates and any party examining the ledger is working from a lie.

R1 *Transactional Atomicity and Isolation:* When a transaction modifies both public and private data, the system must guarantee atomicity and serializable isolation to maintain overall consistency.

R2 *Hash-Data Integrity:* The hash of the private data stored on the replicated ledger must always remain in sync with the actual private data stored in the off-chain collection, acting as a verifiable proof of existence.

(2) Access Control. Privacy guarantees are only as strong as the enforcement beneath them. In a food supply chain, pricing data must be accessible only to the buyer and seller, and access control must be embedded in the same smart contract that governs supply-chain workflows—a separate policy layer creates a seam that can be misconfigured. Equally important: private data must never reach unauthorized peers, even transiently during transaction execution.

R3 *Smart Contract Controlled Access:* Private data in a collection must only be accessible or modifiable via a smart contract, ensuring all data interactions adhere to agreed-upon business processes.

R4 *Authorized Execution Only:* Transactions involving private data must only be executed on peers owned by member organizations of the specified collections, preventing unauthorized peers from seeing or processing private data.

(3) Operational. Consortia are not static. Organizations join and leave, regulations change, and data that must be retained today may need to be purged tomorrow. A new carrier joining GSBN needs access to historical shipping documents; an organization subject to data-retention regulations may need private records purged after a mandated period without disturbing the ledger itself. Private data is also excluded from the consensus-driven block delivery by design, so the system must provide an alternative path through which authorized peers can reliably obtain what they are entitled to hold.

R5 *Dynamic Membership:* Collection membership must be mutable. New members must be able to synchronize historical private data, while removed members should lose access to any data persisted after their removal.

R6 *Data Lifecycle Management:* The system must support a block-to-live (BTL) property for private data, allowing for automatic

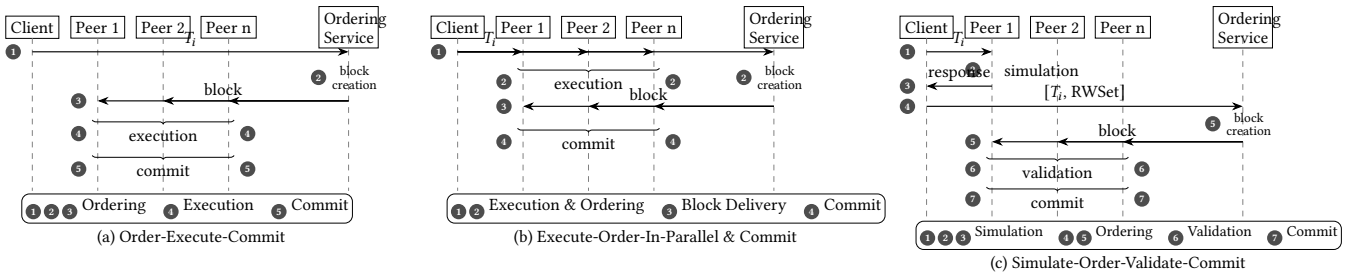

Figure 2: Blockchain replication protocols.

purging to comply with data retention policies or "right to forget" regulations.

R7 *Data Availability:* Since private data is excluded from the consensus-driven block delivery, the system must ensure that all authorized peers can obtain the private data they need before committing a block, even in the presence of network partitions and peer failures.

3.3 Choosing a Blockchain Architecture

To realize these requirements, we must identify a blockchain architecture that natively supports selective execution and data dissemination. In a permissioned blockchain network, a *client* submits transactions on behalf of an application. A *peer* is a node owned by a member organization that maintains a copy of the ledger, executes smart contracts, and validates transactions. An *ordering service* is a decentralized cluster of nodes that establishes the total order of transactions via a consensus protocol. Each peer belongs to exactly one organization, and organizations may operate multiple peers for redundancy. We evaluate three prevalent replication protocols, depicted in Figure 2, to determine which structural properties are necessary for private data support. *Order-Execute-Commit and Execute-Order-In-Parallel & Commit.* Figures 2(a) and (b) show two protocols that differ in timing: the first orders transactions before execution, the second overlaps both. Despite this structural difference, they share a fatal property for our purposes—every node must execute every transaction to maintain ledger consistency. In order-execute-commit (Ethereum [6], Quorum [30], Hyperledger Besu [23]), blocks are ordered by consensus and then broadcast for serial execution across all nodes. The execute-order-in-parallel variant (BlockchainRDBMS [31]) is more aggressive—peers execute and the ordering service sequences concurrently—but all nodes still verify the final block. Restricting execution to a subset of peers in either model would violate state machine replication (SMR) guarantees, breaking R4 (Authorized Execution Only). Every smart contract runs on every node, which means private data touched during execution is visible to the entire network (violating R3), and without a distinct validation phase there is no mechanism to verify private transactions without re-executing them (violating R7).

Simulate-Order-Validate-Commit. Figure 2(c) separates four phases: transactions are speculatively simulated on a configurable subset of nodes, ordered by a consensus service, and then validated via authorization policy checks and OCC [27] based serializability checks before commit. Hyperledger Fabric [1] implements this protocol. Which subset of peers simulates a given transaction is governed by an *endorsement policy*—a predicate specifying the set

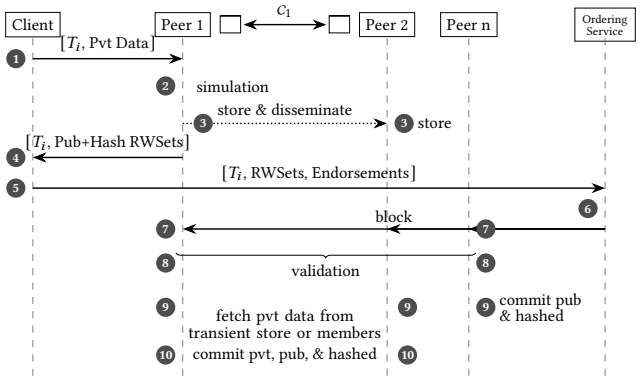

Figure 3: End-to-end private data collection transaction flow. Steps ❶–❹: simulation and dissemination, ❺–❼: ordering, ❽: validation, ❾–❿: commit and retrieval.

of peers whose signatures are required for the transaction to be considered valid. Each endorsing peer independently executes the smart contract and signs the resulting read-write set; the collected signatures are later verified during the validation phase. The key structural property is the separation of simulation from validation: because simulation runs on a subset of nodes dictated by the endorsement policy rather than all of them, we can restrict it to authorized collection members, enabling R4 and R3; the separate validation phase can be augmented to verify hashes of private data on the replicated ledger, enabling R2; the existing OCC-based validation can be extended to cover both public and private state, enabling R1; the decoupling of simulation from consensus creates a natural insertion point for peer-to-peer dissemination, enabling R5 and R7; and because the architecture already separates execution outcomes from the immutable ledger, private data can be stored independently, enabling lifecycle management through purging while retaining on-chain hashes (R6).

4 DESIGN OF PRIVATE DATA TRANSACTIONS

The simulate-order-validate-commit protocol, as argued in the previous section, provides the right foundation for private data. We now describe how private data transaction processing is built on top of it. The design centers on three ideas: *partitioned state management* that splits state into public, hashed, and private domains; *blind validation* that uses the hashed partition as a conflict-complete proxy, enabling serializability checks without accessing private data; and *decoupled dissemination* that separates private data distribution

from block ordering, with proactive push, reactive pull, and background reconciliation ensuring eventual availability. We assume a key-value data model for both public and private state, where each state entry is a $(key, value)$ pair identified by a unique key within a namespace, and each key carries a version that is updated whenever the key is written by a committed transaction. Figure 3 provides an end-to-end overview of the private data collection transaction flow, with numbered steps spanning all four phases; Peer 1 and Peer 2 are members of collection C_1, while Peer n is not. The following subsections detail each component.

4.1 Partitioned State Management

Integrating private data into a shared ledger without exposing it to unauthorized peers during validation is the central challenge. The solution is to partition the state database into three logical domains. Each smart contract is allocated a dedicated *namespace* within these domains; access and modification within a namespace are governed by an endorsement policy specifying the set of authorized peers whose signatures are required to validate state changes. The three partitions for each namespace are:

- *Public State:* Accessible to all nodes; contains the standard replicated ledger state.
- *Hashed State:* Replicated across all nodes; contains cryptographic hashes of private keys and values, providing the evidence required for tamper-detection and serializability checks.
- *Private State:* Isolated to authorized peers; contains the actual private key-value pairs. Modifications are governed by the collection's own endorsement policy $\mathcal{E}$, which supersedes the smart contract's default endorsement policy for private data operations.

Public State and Hashed State are consistent across all peers in the network, providing a globally uniform view of the ledger. Private State, by contrast, varies from peer to peer depending on which collections each peer's organization belongs to—a peer only maintains private state for collections in which it is authorized. By confining all private data interactions to smart contract namespaces, this partitioning satisfies **R3** (Smart Contract Controlled Access); the replicated Hashed State that mirrors Private State reinforces **R2** (Hash-Data Integrity).

4.2 Transaction Simulation

The simulation phase (steps 1–4 in Figure 3) begins when a client submits a transaction proposal to multiple endorsing peers to collect adequate endorsements as per the endorsement policies defined for the smart contract and the private data collections involved. Restricting execution to authorized peers directly satisfies **R4** (Authorized Execution Only). Private data enters the system through a *transient data* field—present during execution but excluded from the final transaction payload, so it is never broadcast to all peers during ordering.

The goal of the simulation phase is to collect read-write sets. Reads capture the data read by the smart contract to make decisions on the proposed changes (i.e., writes). These read-write sets are subsequently used by OCC-based serializability checks in the validation phase. Since state is partitioned into public, hashed, and private domains (§4), the simulation produces three corresponding read-write sets—one per partition: a *public read-write set* for replicated ledger state, a *hashed read-write set* containing cryptographic hashes of private keys and values, and a *private write-set* per collection involved in the transaction. Note that there is no private read-set; private reads are recorded in the hashed read-set via their hash versions to enable blind validation.

Upon receiving the proposal, each endorsing peer executes the smart contract using the APIs defined in Table 2. Public data operations (`GetState`, `PutState`, `DelState`) populate the public read-write set. Private data operations populate the hashed and private partitions: `GetPrivateData`(c, k) records $(H(k), \text{version})$ in the hashed read-set; `PutPrivateData`(c, k, v) performs *dual-recording*, writing (k, v) to the private write-set and simultaneously recording $(H(k), H(v))$ in the hashed write-set, satisfying **R2** (Hash-Data Integrity). `DelPrivateData` and `PurgePrivateData` similarly produce corresponding entries in both the private and hashed write-sets. Table 2 illustrates concretely how a sequence of API calls generates these outputs. When a transaction reads and writes both public and private state, the endorsing peers must satisfy both the smart contract's endorsement policy (for public writes) and the collection's policy $\mathcal{E}$ (for private writes).

Post-Simulation: Transient Storage and Dissemination. Since transaction order is determined only after simulation completes, committing private data to permanent storage at this point would risk corrupting committed state with transactions that are later invalidated. Peers therefore hold private write-sets in a *transient store* until the transaction commits, satisfying **R1** (Transactional Atomicity and Isolation). Entries are keyed by transaction identifier ($txid$) to prevent collisions among concurrently simulated transactions.

As described in Algorithm 1, after simulation the endorsing peer immediately pushes each private write-set to a subset of authorized collection members. The endorser does *not* return its signed response to the client until at least $n_{required}$ peers have acknowledged receipt. Blocking on these acknowledgments guarantees that the data exists on multiple peers before the client can submit the transaction for ordering. After sending the endorsement response, a background gossip task continues pushing the write-set to the remaining peers up to n_{dissem} total, spreading coverage without adding to client-observed latency. We separate $n_{required}$ from n_{dissem} so that operators can tune the two concerns independently. Setting n_{dissem} aggressively ensures most collection members receive the data proactively, which minimizes commit-time reactive pulls; keeping $n_{required}$ small protects endorsement latency. When the endorser cannot obtain $n_{required}$ acknowledgments before the timeout, it returns a failure to the client, who may retry with different endorsers. We deliberately do not retry dissemination at the endorser: keeping the endorsement critical path simple avoids unbounded retry delays and pushes fault handling to the client, where it belongs.

The endorsing peer signs the public and hashed read-write sets and returns them to the client as part of its endorsement response.

Table 2: Smart-contract APIs for public and private data operations, with examples illustrating simulation and blind validation.

API	*State Effect*	*Read-/Write-Set Effect*
`GetState`(k)	Returns value of k from public state	Records (k, version) in public read-set
`PutState`(k, v)	—	Records (k, v) in public write-set
`DelState`(k)	—	Records delete of k in public write-set
`GetPrivateData`(c, k)	Returns value of k from collection c	Records ($H(k)$, version) in hashed read-set
`GetPrivateDataHash`(c, k)	Returns $H(v)$ for key k in collection c	Records ($H(k)$, version) in hashed read-set
`PutPrivateData`(c, k, v)	—	Records (k, v) in private write-set; ($H(k)$, $H(v)$) in hashed write-set
`DelPrivateData`(c, k)	—	Records delete of k in private write-set; delete marker in hashed write-set
`PurgePrivateData`(c, k)	Permanently removes k from local storage	Like `Del`, but purges history to comply with "right to forget" policies

Walkthrough: Simulation and Blind Validation. T_i executes: `GetState`(k_1) → `PutState`(k_1, v_2) → `GetPrivateData`(c_1, k_2) → `GetPrivateData`(c_2, k_3) → `PutPrivateData`(c_1, k_2, v_3) → `PutPrivateData`(c_2, k_3, v_4). Before T_i reaches validation, T_j commits and updates k_2 in c_1 (version v5 → v8). The validator detects the stale read on $H(k_2)$ *without accessing the private value*—blind validation.

Phase	**Public (k_1)**	**Hashed ($H(k_2)$, c_1)**	**Hashed ($H(k_3)$, c_2)**	**Private (k_2, c_1)**	**Private (k_3, c_2)**	**Result**
1. Initial disk	val:v_1, ver:v1	hash:$H(v_5)$, ver:v5	hash:$H(v_5)$, ver:v5	val:v_5	val:v_5	—
2. Simulate T_i	R:v1, W:v_2	R:v5, W:$H(v_3)$	R:v5, W:$H(v_4)$	W:v_3	W:v_4	RW-sets captured
3. T_j commits	—	**ver:v5→v8**, hash:$H(v_8)$	—	val:v_8	—	Disk updated
4. Validate T_i	R(v1)=Disk(v1) ✓	R(v5)≠Disk(v8) ✗	R(v5)=Disk(v5) ✓	No access	No access	T_i **invalid**

Algorithm 1 Proactive dissemination protocol.

Proactive Dissemination Protocol

Input: Transaction tx with private write-sets

At Endorsing Node e (after simulation):
1. Store all private write-sets for tx in local transient store
2. **for each** collection c in tx's private write-sets **do**
   - $W_c \leftarrow$ collection c's private write-set
   - *// Blocking phase: wait for $n_{required}$ ACKs before responding*
   - $P_{req} \leftarrow$ SelectAuthorizedPeers($c.\mathcal{M}, c.n_{required}$)
   - Push $(tx.id, c.id, W_c)$ to all $p_i \in P_{req}$
   - Wait until $|\{\text{org}(p_i) : p_i \text{ ACKed}\}| \geq c.n_{required}$, or timeout
   - **if** timeout **then** return dissemination failure to client
3. Return signed endorsement response to client
4. **for each** collection c in tx's private write-sets **do**
   - *// Background phase: spread to remaining peers up to n_{dissem}*
   - $P_{dissem} \leftarrow$ SelectAuthorizedPeers($c.\mathcal{M}, c.n_{dissem}$) $\setminus P_{req}$
   - Push $(tx.id, c.id, W_c)$ to all $p_i \in P_{dissem}$ (non-blocking)

At Receiving Peer p_i:
1. Verify sender e is authorized for collection c
2. Verify ledger height of tx is within acceptable range
3. Store $(tx.id, c.id, W_c)$ in local transient store
4. Send ACK to endorser e

4.3 Transaction Ordering

Once the client collects sufficient endorsements, it submits the transaction—containing the public and hashed read-write sets, but not the private write-sets—to the ordering service (steps 5–7 in Figure 3). The ordering service is a black box with respect to transaction content: it does not inspect or interpret the read-write sets, and private data is never passed to it. Its sole responsibility is to order transactions into blocks using consensus, and the block order determines the commit order. Depending on deployment requirements, the service can run either a crash fault-tolerant (CFT) protocol such as Raft or a Byzantine fault-tolerant (BFT) consensus protocol. Whether a transaction involves public data, private data, or both, the ordering service's functionality remains unchanged.

4.4 Transaction Validation and Commit

Validation (step 8) and commit (steps 9–10) proceed in two sequential checks for each transaction in a block. The first check verifies the endorser signatures against the applicable endorsement policies; only transactions that pass this check advance to the second, OCC-based serializability check.

Endorsement Policy Validation. Each transaction's endorsement signatures are verified against the smart contract's endorsement policy for public writes and the collection's endorsement policy $\mathcal{E}$ for private writes. A transaction that involves both public and private state must satisfy both policies. Any transaction that fails this check is immediately marked invalid and skipped, satisfying **R4** (Authorized Execution Only). Endorsement policy checks across transactions in a block can run in parallel, since they depend only on the signatures attached to each transaction and not on mutable state.

OCC Serializability Validation. Transactions that pass endorsement validation proceed to OCC-based serializability checks. These checks compare the read-set versions recorded at simulation time against the current committed versions in the Public State and Hashed State. A transaction is serializable if no key in its read-set was modified by an intervening committed transaction. OCC validation proceeds sequentially in block order, since validating one transaction may update state versions visible to subsequent ones. A serialization graph can alternatively enable parallel validation of non-conflicting transactions [29].

Every write to a private key k atomically updates its hash $H(k)$ in the replicated Hashed State, so hashed version identifiers are sufficient to detect all conflicts on private data without exposing the data itself. *Blind validation* follows directly: every peer can verify serializability of private state changes without ever seeing the private data.

Walkthrough. We illustrate blind validation using the example in Table 2. Transaction T_i reads public key k_1 (version v1) and private keys k_2 in collection c_1 and k_3 in collection c_2 (both at version v5), then writes new values for all three. During simulation, the read on k_2 is recorded as $(H(k_2), \text{v5})$ in the hashed read-set—the validator never sees k_2 itself. Before T_i reaches validation, another transaction T_j commits and updates k_2 in c_1, advancing its hashed version from v5 to v8. At validation time, the validator compares T_i's recorded read versions against current disk state: public key k_1 still has version v1 (✓), hashed key $H(k_3)$ in c_2 still has version v5 (✓), but hashed key $H(k_2)$ in c_1 now has version v8, which does not match T_i's recorded version v5 (✗). The validator detects the stale read and marks T_i as invalid—all without accessing any private value. The private columns in Table 2 confirm this: validation requires "No access" to private state.

We now formalize this correctness claim. We adopt the standard OCC serializability criterion [27]: a transaction T_i is *serializable* with respect to a committed history if no key in T_i's read-set was modified by any transaction T_j that committed after T_i began—equivalently, the read-set versions recorded at simulation time must match the current committed versions at validation time. Let $\mathcal{S}_P$ denote the Private State, $\mathcal{S}_H$ the Hashed State, and $\text{ver}(s, k)$ the version of key k in state s.

Theorem 1 (Conflict-Complete Proxy). *Assuming H is a collision-resistant hash function (so that $H(k_1) = H(k_2) \implies k_1 = k_2$), the Hashed State $\mathcal{S}_H$ is a conflict-complete proxy for the Private State $\mathcal{S}_P$: for any transaction T_i involving private data, an OCC conflict exists in $\mathcal{S}_P$ if and only if a corresponding conflict exists in $\mathcal{S}_H$. Formally, for every private key k in collection c:*

$$\text{ver}(\mathcal{S}_P, k) \neq \text{ver}_{\text{sim}}(T_i, k) \iff \text{ver}(\mathcal{S}_H, H(k)) \neq \text{ver}_{\text{sim}}(T_i, H(k))$$

Proof. We establish three invariants maintained by the API (Table 2):

(1) *Write Bijection:* By construction, every `PutPrivateData`(c, k, v) atomically produces both a private write-set entry (k, v) and a hashed write-set entry $(H(k), H(v))$. The commit procedure assigns both entries the same version identifier (block number, transaction index). `DelPrivateData` and `PurgePrivateData` similarly produce corresponding hashed entries. Thus, $\text{ver}(\mathcal{S}_P, k)$ and $\text{ver}(\mathcal{S}_H, H(k))$ are updated in lockstep: $\text{ver}(\mathcal{S}_P, k) = \text{ver}(\mathcal{S}_H, H(k))$ after every committed transaction.

(2) *Read Recording:* Every `GetPrivateData`(c, k) records $\text{ver}(\mathcal{S}_H, H(k))$ in the hashed read-set. Since Invariant 1 guarantees $\text{ver}(\mathcal{S}_P, k) = \text{ver}(\mathcal{S}_H, H(k))$, this recorded version is identical to the version that would appear in a hypothetical private read-set.

(3) *Conflict Detection:* During OCC validation, a conflict on private key k exists iff $\text{ver}(\mathcal{S}_P, k)$ has changed since simulation. By Invariant 1, $\text{ver}(\mathcal{S}_P, k)$ changes iff $\text{ver}(\mathcal{S}_H, H(k))$ changes. By Invariant 2, the validator compares $\text{ver}(\mathcal{S}_H, H(k))$ against the same baseline. Therefore, the OCC check on $\mathcal{S}_H$ detects a conflict iff the corresponding check on $\mathcal{S}_P$ would.

Since both directions hold, blind validation produces neither false negatives (missed conflicts) nor false positives (spurious aborts). □

Edge Case: Concurrent Membership Changes. A collection's membership policy $\mathcal{M}$ may be updated in the same block that contains private data transactions for that collection. Whether processed sequentially or via a serialization graph [29], a membership change at transaction index j takes effect for all subsequent transactions $j' > j$ within the same block. This within-block ordering is a deliberate design choice: it avoids the complexity of batch-level policy resolution and guarantees deterministic validation across all peers. Endorsement and membership checks during validation therefore always reflect the correct policy version. A newly added member will not yet have historical private data at validation time; the reactive retrieval and reconciliation mechanisms (§4.5) handle this asynchronously without affecting current-block validity. Theorem 1 and the sequential policy application ensure **R1** across public and private state. Once a transaction is successfully validated, the private write-sets are moved from the transient store to the permanent private data store. The Private State within a smart contract's namespace reflects only the latest value for each key, overwritten by each subsequent committed transaction. To support historical retrieval, a separate *private data store* retains all historical private write-sets indexed by block number and transaction index, so that when a new member joins a collection, the complete history of private data can be synchronized to that member.

Reactive Retrieval of Missing Private Data. A peer may not possess all private write-sets at commit time—for example, due to network delays during proactive dissemination. Prior to committing a block, each peer identifies private data for which it is an authorized collection member but has not yet received, and retrieves it using the prioritized pull strategy in Algorithm 2. The protocol targets the transaction's endorsers first, since they are guaranteed to possess the write-sets, having produced them during simulation. Only if the endorsers are unavailable does the peer fall back to broadcasting across the broader gossip network. Each received write-set is verified against the on-chain hash before being stored, so a Byzantine peer cannot inject forged data through this path. If retrieval succeeds within the stipulated timeout, the peer commits the block with complete private data, satisfying **R7** (Data Availability).

Algorithm 2 Reactive retrieval protocol.

Reactive Retrieval Protocol

Input: Block B with hashed write-sets, local transient store

At Committing Peer p (before commit):

1. **for each** transaction $tx \in B$ **do**
   if tx fails endorsement policy **then skip**
2. **for each** collection c referenced in tx's hashed write-set **do**
   if $p \in c.\mathcal{M}$ **and** c's private write-set $\notin$ transient store
   Add digest $(tx.id, c.ns, c.id, B.num, tx.idx)$ to MissingSet
   // $c.ns$: smart contract namespace owning collection c
3. **if** MissingSet $= \emptyset$ **then** proceed to commit
4. $E \leftarrow$ endorsers of tx *// priority targets*
5. Send pull request with MissingSet digests to E
6. **for each** private write-set W_c received **do**
   Verify $H(W_c)$ matches on-chain hash; if valid, store W_c
7. **if** MissingSet $\neq \emptyset$ **then**
   Broadcast to gossip network; retry with fixed-interval sleep
8. Proceed to commit (missing data tracked for reconciliation)

Should reactive retrieval fail to pull the data within the stipulated time, the peer commits the block anyway to avoid blocking

progress and preserve liveness. The missing entries are recorded as *eligible missing data*—the block number, transaction index, collection identifier, and the corresponding hashed key-value pairs from the hashed write-set are stored so that a background reconciliation service can periodically retry retrieval, and the stored hashes allow the peer to verify the authenticity of data received later. Until reconciliation completes, any `GetPrivateData` call for a missing key returns an error, signaling to the smart contract that the data is not yet locally available. Private data for collections in which the peer's organization is *not* a member is always tracked as *ineligible missing data*. These entries are never fetched, but are retained because if the organization later joins the collection, they can be used to identify and synchronize historical private data the peer is now authorized to access, supporting **R5** (Dynamic Membership).

4.5 Dynamic Membership, Reconciliation, and Data Lifecycle

Reconciliation and Dynamic Membership. A background reconciliation service periodically scans the eligible missing data recorded at commit time and fetches the corresponding private write-sets from authorized collection members, using the same hash-based verification as Algorithm 2. This ensures eventual data availability without blocking block processing, satisfying **R7** (Data Availability). When a new member is added to a collection's membership policy $\mathcal{M}$, the peer detects the configuration change during the commit of the block containing the policy update. At that moment, all *ineligible missing data* for that collection is reclassified as *eligible*, automatically triggering reconciliation to pull historical private write-sets from existing members (**R5**). Membership removal is handled symmetrically: the departing peer retains access to data committed before its removal, but the updated policy excludes it from both proactive dissemination and reactive retrieval eligibility checks, preventing any subsequent private data from reaching that organization.

Data Lifecycle Management. The private data store supports two complementary mechanisms for data lifecycle management, satisfying **R6** (Data Lifecycle Management).

Automatic expiry. Each collection defines a block-to-live (BTL) retention parameter τ. Any private data committed more than τ blocks ago is considered expired. A background process periodically purges expired private data while preserving the on-chain hashes, keeping the historical audit trail tamper-evident.

Explicit purge. For finer-grained control—e.g., to comply with "Right to Forget" regulations on a per-key basis—the `PurgePrivateData(c,k)` API allows a smart contract to permanently remove a specific key from both the current private state and all historical versions stored in the private data store. This is distinct from `DelPrivateData`, which only removes the key from the current state while retaining historical write-sets. When a purge is committed, the peer records a *purge marker* keyed by $(c, H(k))$ that permanently indicates the key has been purged at a given block and transaction index. A *hashed index* maps each hashed key to the block and transaction locations of its historical write-sets, enabling efficient background cleanup without scanning entire write-sets.

Purge interacts with the reconciliation protocol: when a peer responds to a reconciliation or reactive retrieval request, it filters out keys that have been purged locally, returning a partial write-set rather than a complete one. The requesting peer accepts partial write-sets by verifying individual key hashes rather than a single hash over the entire collection write-set. If a requesting peer receives data for a key it has already purged, it discards the data. Conversely, if a peer requests data that the responder has already purged, the responder omits those keys and the requester re-queues the request until it observes the purge transaction in a committed block, at which point it removes the entry from its missing data set.

In both mechanisms, on-chain hashes are preserved after private data removal, keeping the historical audit trail tamper-evident. The implications of retaining these on-chain hashes are discussed in §5.

5 THREAT MODEL AND SECURITY ANALYSIS

Our design assumes a decentralized network of organizations in which some nodes may be faulty or malicious. We characterize the trust assumptions for each entity class and the security implications that follow.

Peer Nodes. For *correctness*, the endorsement policy is the trust boundary for hash-data integrity (R2). Each endorsing peer independently computes the hash of private data during simulation and signs the result; since the endorsement policy requires matching responses from peers across multiple organizations, no single compromised peer can record a fraudulent hash or unilaterally influence transaction outcomes. However, if all endorsing members of a collection collude, they can commit a hash that does not correspond to the actual private data, breaking R2. This is an inherent limitation: the system guarantees hash-data integrity only under the assumption that the endorsement policy holds—i.e., at least one endorser is honest. *Confidentiality* is more nuanced. A malicious peer can leak private data it legitimately holds, or refuse to disseminate data to eligible peers; these are inherent limitations of access-control-based privacy, which prevents unauthorized access but cannot constrain how an authorized party uses data it possesses. Confidentiality is therefore bounded by the weakest collection member. Purging offers no stronger assurance: a malicious member can retain private data locally without honoring purge requests, so purge is best-effort rather than a cryptographic guarantee.

Clients. Clients may be malicious, submitting inconsistent private data to different endorsers, fabricating simulation results without sufficient endorsements, or flooding the system with orphan write-sets. The hash mechanism neutralizes inconsistent submissions: if a client sends different private data to different endorsers, each endorser computes a different hash and includes it in its signed response; because the endorsement policy requires matching responses from multiple endorsers, the divergent hashes render the endorsements incompatible and the transaction is rejected during validation. Fabricated simulations are similarly neutralized—peers perform endorsement policy validation *before* attempting to pull any missing pre-images, discarding invalid transactions without fetching their data. For write-set flooding, where a client simulates many transactions but never submits them, a retention window Δ bounds how long orphan entries persist: the transient store automatically purges entries whose simulation height lags the current ledger height by more than Δ blocks, and receiving peers reject

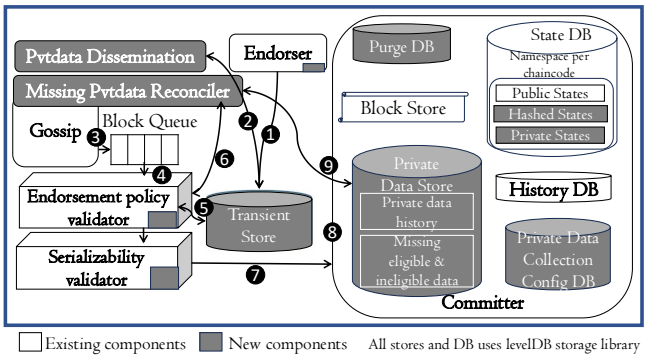

Figure 4: Peer architecture with private data collection components. Pre-existing components (white) form the core transaction pipeline; new components (gray) implement the private data collection mechanism.

disseminated write-sets outside this window. The same retention window defends against a malicious *peer* flooding fake write-sets to other collection members; such activity is tracked through monitoring metrics that allow operators to identify and remediate faulty peers.

Other Considerations. Only public and hashed read-write sets are submitted to the ordering service, so ordering nodes never see private data payloads; the ordering layer supports both CFT (Raft) and BFT consensus. We assume a partially synchronous network in which partitions are possible but eventually resolve—a partition may delay private data dissemination, but a peer that cannot obtain private data before commit records it as missing and reconciles once connectivity is restored. Private data must be salted by the client before hashing to prevent adversaries from predicting or confirming the underlying data through brute-force hash comparison. Regarding data lifecycle, purging removes private data from the off-chain store but the on-chain hash is retained on the immutable ledger; this may not fully satisfy regulations such as GDPR that require complete erasure, and we expect peers to be honest in executing purge operations. Finally, private data collections protect payload content but not transaction *metadata*: the hashed read-write sets recorded on the public ledger reference specific collections, and collection membership policies are visible to all network participants, so any peer can infer which organizations are party to a private transaction even without seeing the underlying data. Mitigating this leakage would require techniques such as zero-knowledge set membership proofs [7] or anonymous credentials [8].

6 IMPLEMENTATION

The private data collection mechanism is designed for any blockchain platform that follows the simulate-order-validate-commit replication protocol. We implemented it within Hyperledger Fabric [1], comprising approximately 33,000 lines of Go code (production logic and tests), shipped in production since v1.2. Figure 4 shows how the new components (gray) extend the existing peer architecture (white).

Private Data Flow. During simulation, the *endorser* writes private write-sets to the *transient store* (❶), a temporary staging area keyed by transaction ID, while the *Pvtdata Dissemination* component concurrently pushes private write-sets to authorized collection members (❷). When the *gossip* layer delivers an ordered block from the ordering service into the *block queue* (❸), the *endorsement policy validator* checks each transaction against both chaincode-level and collection-level policies (❹). For valid transactions, the peer retrieves private data from its local *transient store* (❺); any data still missing—due to network delays—is pulled from eligible peers by the *Missing Pvtdata Reconciler* (❻). The *serializability validator* then performs blind OCC validation against the public and hashed state partition in the *State DB*, verifying serializability without accessing private data (Theorem 1) (❼). Finally, the *committer* moves validated private data from the transient store to the durable *Private Data Store* (❽), which also tracks eligible and ineligible missing data for future reconciliation, and updates the public, hashed, and private partitions in the *State DB* (❾), where per-chaincode namespace isolation prevents cross-contract access to hashed or private state. A background *Purge DB* process expires private data based on the block-to-live parameter τ, and the `PurgePrivateData` API (Table 2) supports application-controlled deletion by committing purge markers that prevent data from appearing in queries or being reconciled, providing the "right to forget" behavior required by data protection regulations.

Commit-Path Optimizations. Initial microbenchmarks of the commit path revealed a 50% throughput penalty for PDCs (720 vs. 1440 tx/s). Profiling identified cryptographic membership checks and serial disk I/O as the dominant bottlenecks. We applied a series of crash-safe optimizations (Table 3) that reduced this overhead to just 6%. Key techniques include replacing the commit-time collection membership check—which originally invoked the full policy evaluation engine with certificate deserialization and signature verification—with an O(1) organization identifier lookup (safe because cryptographic policy enforcement already occurs at endorsement and validation time), parallelizing block-store and private-data-store commits, deferring transient-store purge to a background routine, and adding a transient-store read cache.

7 EVALUATION

We evaluate four questions: (1) what is the end-to-end overhead of private data relative to all-public transactions? (2) how sensitive is performance to dissemination parameters? (3) how far does the system scale with collection count? and (4) how resilient is the system to membership changes and faults?

Experimental Setup. We design four application workloads modeling multi-party business processes to stress different dimensions of the PDC mechanism (Table 4). Supply Chain maximizes the collection count per organization; Auction isolates dissemination throughput; Trade Finance stresses scalability under strict majority endorsement; and EHR tests read throughput. For each, we compare a *PDC variant* against an *all-public variant* to isolate privacy overhead. In addition, we develop `PDC_BENCH`, a minimal chaincode for controlled micro-benchmarking that exercises all public and private data APIs, enabling direct measurement of individual PDC operations against a public-ledger baseline.

Table 3: Commit-path optimizations.

Optimization Step	tx/s	Gain
PDC baseline (No optimizations)	720	—
+ O(1) membership identifier check	880	+22%
+ Background transient-store purge	1040	+44%
+ Parallel block and private data store commit	1152	+60%
+ Parallel purge and state DB commit	1248	+73%
+ Transient-store cache	1344	+87%
Public data only (Reference baseline)	1440	—

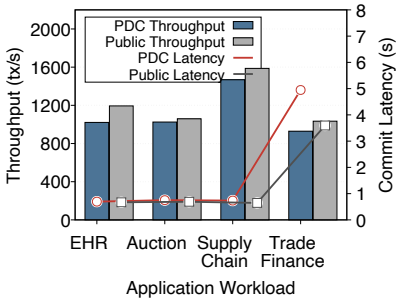

Figure 5: End-to-end throughput and commit latency for PDC and all-public.

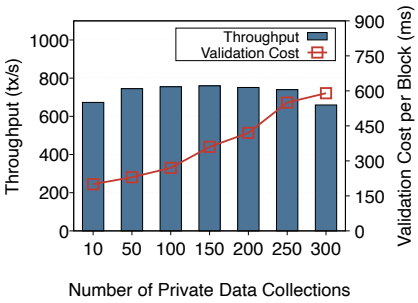

Figure 6: Throughput and per-block validation cost as collections scale to 300.

Table 4: Workload characteristics and experimental setup.

	Supply Chain	Auction	Trade Finance	EHR
Domain	Farm-to-retail	Sealed-bid auction	Int'l trade	Patient care
Orgs / Peers	10 / 10	6 / 6	12 / 12	8 / 8
PDC collections	17	9	21	13
Only-PDC tx	—	48.1%	45.9%	0.0%
Mixed tx	—	0.0%	0.0%	48.6%
Public-only tx	—	51.9%	54.1%	51.4%
Endorsement policy	OR (any 1)	AND (2/2)	Majority (7/12)	2-of-4 roles
Reads / Writes	1 / 1	2 / 1	1 / 2	3 / 1
VMs†	14	10	16	12

†Each VM: 32 vCPUs (2x8-core Xeon), 64 GB RAM, 12 TB disk, RHEL 9 / Raft orderers. VMs span 4 WAN regions (US-E, US-S, EU, JP). Load is increased until saturation; we report peak tx/s and latency over 300 s.

7.1 End-to-End Performance

We compare the PDC and all-public variants of all four workloads at peak throughput, with roughly 40–50% of transactions touching private data collections in each PDC variant. Figure 5 shows throughput and commit latency side by side. Across all four workloads, PDCs consistently reduce peak throughput: 3% for Auction, 7.5% for Supply Chain, 10% for Trade Finance, and 14% for EHR. The overhead correlates with per-transaction complexity. EHR sees the largest drop because it is the most read-heavy workload (3 reads per tx) and each private-data read adds a private-data-store lookup. Trade Finance suffers from its strict 7-of-12 endorsement policy: the additional dissemination and commit-path work compounds an already long critical path. Auction and Supply Chain, with simpler profiles and lighter endorsement policies, see the smallest overheads. Commit latency increases by 4–17% across the workloads, consistent with the additional hash verification and private-data-store operations on the commit path.

Where does the overhead originate? Per-node Prometheus metrics and disk I/O counters collected during each 300 s run point to three sources. (1) *CPU:* per-transaction cost rises 10–20% due to hash computation, transient-store writes, and private-data-store flushes—none of which exist in the all-public path. (2) *Network:* because the ordering service never sees private payloads, they must be distributed point-to-point via gossip, increasing per-transaction traffic by 10–15%; missed payloads trigger reactive retrieval that further inflates peer-to-peer traffic. (3) *Disk reads:* committers query the transient store and private-data store to match payloads against on-chain hashes, increasing read I/O by 2–4×. Notably, total disk *writes* decrease: in the all-public path every peer writes the full payload, whereas with PDCs the ledger records only a hash and only authorized peers persist the private payload—a storage benefit of selective replication.

7.2 Scalability

To stress-test the per-collection overhead, we deploy a 10-organization WAN network and scale the number of private data collections attached to a single chaincode from 10 to 300 in increments of 50 (configurations c10–c300). The workload uses `PDC_BENCH`: each transaction writes to a single randomly selected collection; all other parameters (block size, endorsement policy, value size) remain fixed. Figure 6 plots throughput (bars) and per-block validation cost (line) as the collection count grows. The system sustains peak throughput (740–760 tx/s) up to 250 collections. Starting at c150, average transaction latency begins to stretch; at c300, throughput drops by 11% (to 660 tx/s) while average latency spikes to 24 s. Disk I/O, ledger commit time, and state DB metrics remain flat throughout the range, confirming that the storage layer is *not* the bottleneck. Instead, the dominant cost is CPU time spent in the validation phase: per-block validation cost rises linearly from 0.20 s at c10 to 0.59 s at c300 (Figure 6).

This degradation is *not* an architectural limit but an implementation artifact. The root cause is a serialization shortcut: all collection definitions are stored in a single protobuf package per chaincode, and the validator fully deserializes this package for every transaction that touches any collection—even though each transaction touches only one. At 300 collections, this deserialization runs hundreds of times per second, exhausting CPU headroom. Caching the parsed package or indexing individual collection policies would eliminate the linear penalty without architectural changes. Real-world deployments today rarely exceed a few dozen collections per chaincode, well within the flat-throughput region.

7.3 Dissemination Sensitivity

To isolate the effect of the dissemination parameters introduced in Algorithm 1, we deploy a 20-organization WAN network where all

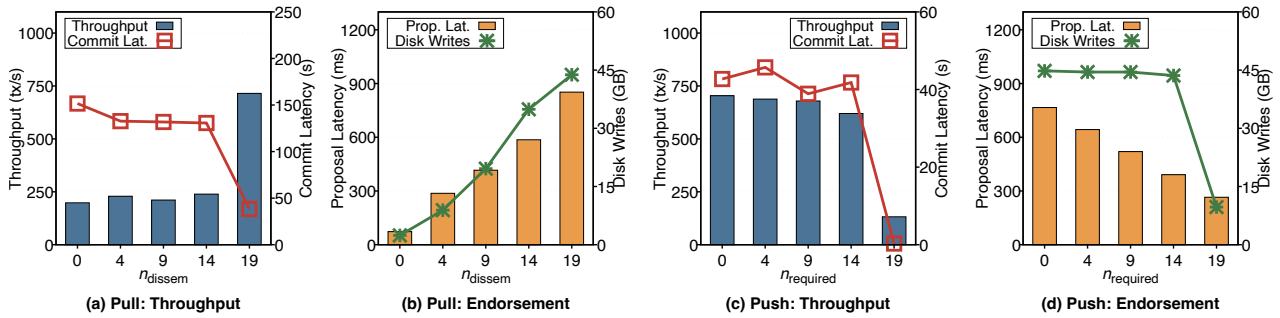

Figure 7: Dissemination parameter sensitivity on a 20-organization WAN network: pull-dominant (a, b) varying n_{dissem} and push-with-quorum (c, d) varying $n_{required}$.

peers share a single private data collection. The workload consists entirely of private-data write transactions, each requiring a single endorsement, running at 600 tx/s for 300 s. We study two dissemination strategy families by varying the collection configuration parameters n_{dissem} and $n_{required}$:

- *Pull-dominant:* The endorsing peer proactively pushes to $n_{dissem} \in \{0, 4, 9, 14, 19\}$ peers before ordering; the remaining $20 - n_{dissem}$ peers retrieve private data reactively at commit time ($n_{required} = 0$, so endorsement latency is unaffected by dissemination).
- *Push-with-quorum:* The endorsing peer pushes to all 20 peers ($n_{dissem} = 20$) but blocks the endorsement response until $n_{required} \in \{0, 4, 9, 14, 19\}$ peers acknowledge receipt.

Figure 7 shows performance under the two strategies. Under pull-dominant dissemination, throughput degrades as n_{dissem} decreases (Figure 7(a)): with $n_{dissem} = 0$, only the endorsing peer holds the private data at commit time, so all 19 remaining peers issue pull requests. Because pulls target random peers, most miss the single data holder and time out. Transactions still commit—peers record the missing data for background reconciliation—but the low cumulative disk writes in Figure 7(b) confirm that most peers have not yet persisted the private data within the measurement window.

Under push-with-quorum, all private data reaches every peer (disk writes remain uniformly high in Figure 7(d)), but requiring WAN acknowledgments before returning the endorsement response creates backpressure. Figure 7(c) shows throughput dropping 81% at $n_{required} = 19$ as the endorser waits for the slowest cross-region peer. Proposal latency rises correspondingly (Figure 7(d)), confirming that the bottleneck shifts from commit-time retrieval to endorsement-time blocking. We conclude that asynchronous push ($n_{dissem} = N-1, n_{required} = 0$) is the optimal WAN configuration: it ensures all peers receive private data before commit time (full disk writes across all peers), maximizes throughput, and eliminates commit-time pull storms and deferred reconciliation.

7.4 Dynamic Membership and Fault Tolerance

We design a two-phase experiment on a five-organization Fabric network (one peer per org). In Phase 1, four organizations populate a shared private data collection with 50,000 key-value pairs. In Phase 2, the collection policy is expanded to a fifth organization via a chaincode upgrade; the new peer reconciles the 50,000 historical

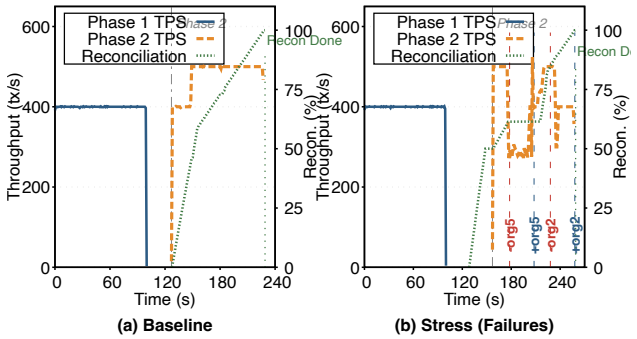

Figure 8: Dynamic membership and fault tolerance. (a) Baseline. (b) Stress with compound peer failures and recoveries.

records while an additional 50,000 transactions execute concurrently. We run this under two conditions: (i) a *baseline* with no faults, and (ii) a *stress* configuration that crashes the newly joined peer mid-reconciliation and an existing data-holder peer, each followed by recovery after a delay. Correctness is verified by fetching all 100,000 key-value pairs from every peer after stabilization.

In the baseline (Figure 8(a)), throughput rises from 396 tx/s to 474 tx/s when the fifth organization joins, and reconciliation of 50,000 historical records completes at $t \approx 229$ s without visible throughput degradation, confirming that background synchronization is effectively isolated from the transaction pipeline. Under stress (Figure 8(b)), crashing the synchronizing peer (`-org5`) halts reconciliation immediately, but it resumes from its last checkpoint upon recovery (`+org5`). Crashing an existing data-holder (`-org2`) drops throughput from ~500 tx/s to the 4-peer baseline of ~400 tx/s. Despite compound failures, reconciliation reaches 100% data consistency at $t \approx 259$ s, confirming that the gossip-based pull mechanism is self-healing.

8 REAL-WORLD IMPACT

Since its introduction in Hyperledger Fabric v1.2, the private data collection mechanism has seen broad adoption across industries with distinct privacy requirements. MineHub [24] deploys PDCs to isolate bilateral contract data across thousands of supply-chain

participants in mining and metals, replacing the coarse-grained channel model that could not scale to dynamic data-sharing relationships. The IBM Food Trust—deployed at Walmart and other major retailers [22, 42]—protects supplier-specific pricing and process data with PDCs while preserving end-to-end traceability for food safety. GSBN (Global Shipping Business Network) [25] relies on PDCs to keep private trade documents such as Bills of Lading confidential within its permissioned shipping network.

The pharmaceutical sector illustrates a particularly instructive use case. The MediLedger project [38, 40] combines PDCs with zero-knowledge proofs for drug tracking under the U.S. Drug Supply Chain Security Act (DSCSA), an approach validated by an FDA pilot program; this confirms that PDCs can serve as a substrate for layering additional cryptographic guarantees when the threat model demands them. In aerospace, Honeywell's GoDirect Trade marketplace [21] uses both channels and private data collections to isolate transaction data in its B2B marketplace for used aircraft parts. In financial services, Oracle's Blockchain Platform [33] stores bank account numbers solely in the private data of participating organizations while recording one-way hashes on the shared ledger, protecting account attributes in a wholesale Central Bank Digital Currency (CBDC) solution.

Across all of these deployments the underlying pattern is the same: PDCs partition sensitive bilateral data—pricing, contracts, account identifiers—while preserving a shared, hash-verified audit trail that satisfies regulatory compliance and cross-party verification requirements.

9 RELATED WORK

Channels and Sub-Networks. Hyperledger Fabric's *channels* [1] provide data isolation via independent blockchain instances, each with its own ledger and consensus. While effective, channels are coarse-grained: each data-sharing relationship may require a separate channel, leading to operational complexity. Our mechanism provides fine-grained privacy within one channel.

Private Transactions via Off-Chain Managers. Quorum [30] and Hyperledger Besu [23] both implement a hash-on-chain / data-off-chain model using an external transaction manager (Tessera). However, both have static privacy group membership, require all participants to be online, and offer limited interoperability between private and public state within a single transaction. Our mechanism supports dynamic membership with retroactive reconciliation, tolerates offline peers via gossip, and enables atomic cross-state reads and writes. R3 Corda [4] eliminates the global ledger entirely, sharing transactions only on a need-to-know basis. This introduces a *backchain resolution* problem [4]: to verify an input state, a recipient must walk the chain of prior transactions that produced it, and each step in this chain may involve data from unrelated parties who never consented to its disclosure. The result is that verifying provenance can inadvertently expose historical data to new counterparties. That same architectural choice also precludes network-wide queries and the tamper-evident audit trail that our on-chain hashes provide.

Cryptographic Privacy. Zero-knowledge proof systems provide strong privacy guarantees: Zerocash [2] uses zk-SNARKs [3] for fully anonymous payments; Hawk [26] extends this to smart contracts but requires a trusted manager; Zether [5] enables confidential token transfers via ElGamal encryption and range proofs. All incur significant computational cost (seconds per proof), making them unsuitable for high-throughput enterprise workloads. Our work is complementary—MediLedger [38, 40] demonstrates combining PDCs with ZKPs in practice. Confidential Transactions [28], based on Pedersen commitments [34], hide amounts via homomorphic commitments but not sender/receiver identities. CryptoNote [41] (Monero) combines ring signatures and stealth addresses for anonymity. These target permissionless cryptocurrency privacy—a different problem from selective access control in permissioned networks.

TEEs, MPC, and Homomorphic Encryption. TEE-based systems such as Ekiden [9] and CCF [37] execute smart contracts inside hardware enclaves (e.g., Intel SGX), but introduce hardware dependencies and side-channel vulnerabilities. MPC platforms like Enigma [43] enable joint computation over private inputs, but performance overhead led the project to pivot to TEEs. Fully homomorphic encryption [19] allows computation on encrypted data but incurs 10^4–$10^6\times$ overhead. Our approach sidesteps these costs: authorized peers process private data in plaintext while the blockchain stores only hashes, requiring no specialized hardware.

Off-Chain State Anchoring. State channels (e.g., Lightning [35]) and rollups (Optimistic [11], ZK [12]) share the architectural pattern of anchoring off-chain state on a base ledger, but address a different problem: scalability in permissionless networks. Both assume a public verification layer—fraud proofs or validity proofs—that any party can challenge, whereas PDCs restrict both data visibility and verification to authorized collection members. The anchoring mechanism also differs: rollups post compressed transaction data or proofs for global verifiability, while PDCs record only hashes and rely on blind validation for serializability, since the goal is privacy rather than throughput scaling.

Database-Layer Privacy. CryptDB [36] executes SQL over encrypted data with practical overhead (15–26%) but progressively leaks information. Differential privacy [10] provides statistical guarantees against individual record disclosure. These complement our work: PDCs control *who* accesses data at the replication layer, while database techniques limit *what* queries reveal.

Positioning. The distinguishing property of our mechanism is that it achieves fine-grained privacy through architecture—selective replication, on-chain hashes, and blind validation—rather than through expensive cryptographic primitives or hardware trust assumptions. Each alternative surveyed above sacrifices at least one desirable property: ledger-level isolation forfeits fine granularity, eliminating the global ledger forfeits a network-wide audit trail, proof-based confidentiality forfeits throughput, and hardware enclaves forfeit deployment flexibility. PDCs avoid these trade-offs by processing private data in plaintext on authorized peers and recording only hashes on the shared ledger, preserving auditability, atomicity, and smart-contract expressiveness simultaneously.

10 CONCLUSION

We presented *private data collections* (PDCs): private data is stored only on authorized peers while its cryptographic hash is recorded on the replicated ledger. A transient store, hashed read-write sets,

and a gossip-based dissemination protocol together deliver privacy, atomicity, integrity, and auditability within a single shared ledger—without specialized hardware, expensive cryptographic primitives, or the management overhead that makes channel-based deployments brittle at scale. Our Hyperledger Fabric implementation has been production-ready since v1.2 and achieves throughput and latency comparable to standard public transactions. Deployments at Walmart, MediLedger, GSBN, and MineHub confirm that the design generalizes across industries with very different trust models and data-sharing patterns. More broadly, PDCs demonstrate that selective transparency—giving designated parties verifiable access to exactly the data they are authorized to see—is achievable within a standard permissioned blockchain architecture, opening a practical path for regulated industries that have so far found blockchain's transparency model incompatible with their compliance obligations.

REFERENCES

[1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, pages 30:1–30:15, New York, NY, USA, 2018. ACM.

[2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy (SP)*, pages 459–474. IEEE, 2014.

[3] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *Proceedings of the 23rd USENIX Conference on Security Symposium*, SEC'14, page 781–796, USA, 2014. USENIX Association.

[4] R. G. Brown, J. Carlyle, I. Grigg, and M. Hearn. Corda: An introduction. R3 Technical White Paper, 2016.

[5] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh. Zether: Towards privacy in a smart contract world. *IACR Cryptology ePrint Archive*, 2019:191, 2019.

[6] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform, 2014. Accessed: July 31, 2019.

[7] J. Camenisch, R. Chaabouni, and a. shelat. Efficient protocols for set membership and range proofs. In *Advances in Cryptology — ASIACRYPT 2008*, pages 234–252. Springer, 2008.

[8] J. Camenisch and A. Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology — EUROCRYPT 2001*, pages 93–118. Springer, 2001.

[9] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *Proceedings of the IEEE European Symposium on Security and Privacy*, EuroS&P '19, pages 185–200. IEEE, 2019.

[10] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[11] Ethereum Foundation. Optimistic rollups. https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/, 2023.

[12] Ethereum Foundation. ZK-rollups. https://ethereum.org/en/developers/docs/scaling/zk-rollups/, 2023.

[13] European Commission. Food safety policy. https://food.ec.europa.eu/horizontal-topics/general-food-law_en, 2023.

[14] European Parliament and Council of the European Union. Regulation (EC) no 178/2002 – general food law. https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=celex:32002R0178, 2002.

[15] European Parliament and Council of the European Union. General data protection regulation (GDPR) – regulation (EU) 2016/679, 2016.

[16] Food and Agriculture Organization of the United Nations. Food traceability guidance. http://www.fao.org/3/a-i7665e.pdf, 2017.

[17] Food Standards Australia New Zealand. Food standards Australia New Zealand. https://www.foodstandards.gov.au/, 2023.

[18] J. S. A. G. C. N. Gaur. *Blockchain for Business*. Addison-Wesley Professional, paperback edition, 2019.

[19] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, pages 169–178, New York, NY, USA, 2009. ACM.

[20] HHS. Hipaa privacy rule.

[21] Hyperledger Foundation. Case study: Honeywell aerospace creates online parts marketplace with Hyperledger Fabric. https://www.lfdecentralizedtrust.org/case-studies/honeywell-case-study, 2019.

[22] Hyperledger Foundation. How Walmart brought unprecedented transparency to the food supply chain with Hyperledger Fabric. https://web.archive.org/web/20210308235803/https://www.hyperledger.org/wp-content/uploads/2019/02/Hyperledger_CaseStudy_Walmart_Printable_V4.pdf, 2019.

[23] Hyperledger Foundation. Hyperledger Besu. https://www.lfdecentralizedtrust.org/projects/besu, 2020.

[24] Hyperledger Foundation. Case study: Managing the metal and mining industry's supply chain. https://www.lfdecentralizedtrust.org/case-studies/krypc-minehub-case-study, 2021.

[25] Hyperledger Foundation. GSBN simplifies global trade with Hyperledger Fabric. https://www.lfdecentralizedtrust.org/case-studies/gsbn-case-study, 2022.

[26] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 839–858, 2016.

[27] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2):213–226, June 1981.

[28] G. Maxwell. Confidential transactions, 2015.

[29] H. Meir, A. Barger, and Y. Manevich. Increasing concurrency in hyperledger fabric. In *Proceedings of the 12th ACM International Conference on Systems and Storage*, SYSTOR '19, pages 179–179, New York, NY, USA, 2019. ACM.

[30] J. Morgan-Chase. Quorum private transactions.

[31] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran. Blockchain meets database: Design and implementation of a blockchain relational database. *Proc. VLDB Endow.*, 12(11):1539–1552, July 2019.

[32] National Center for Biotechnology Information. Escherichia coli. https://www.ncbi.nlm.nih.gov/books/NBK507845/, 2023.

[33] Oracle. Robust, scalable and efficient system for confidential token transactions: Oracle's breakthrough on Hyperledger Fabric. https://docs.oracle.com/en/cloud/paas/blockchain-cloud/digitalassetsoci/confidential-transactions.html, 2024.

[34] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In J. Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, pages 129–140, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

[35] J. Poon and T. Dryja. The Bitcoin lightning network: Scalable off-chain instant payments. https://lightning.network/lightning-network-paper.pdf, 2016.

[36] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan. Cryptdb: Protecting confidentiality with encrypted query processing. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, SOSP '11, pages 85–100, New York, NY, USA, 2011. ACM.

[37] M. Russinovich, E. Ashton, C. Avanessians, M. Castro, A. Chamayou, S. Clebsch, M. Costa, C. Fournet, M. Kerner, S. Krishna, J. Maffre, T. Shermer, and O. Roy. CCF: A framework for building confidential verifiable replicated services. Technical Report MSR-TR-2019-16, Microsoft Research, 2019.

[38] B. Taylor. Hyperledger for healthcare: How Fabric drives the next-generation pharma supply chain. https://www.lfdecentralizedtrust.org/blog/2019/12/02/hyperledger-for-healthcare-how-fabric-drives-the-next-generation-pharma-supply-chain, 2019.

[39] U.S. Congress. Drug supply chain security act (DSCSA), title II of the drug quality and security act. https://www.fda.gov/media/168283/download, 2013.

[40] U.S. Food and Drug Administration. DSCSA pilot project program: MediLedger final report. https://www.fda.gov/media/168283/download, 2023.

[41] N. van Saberhagen. CryptoNote v 2.0. Whitepaper, 2013.

[42] Walmart Inc. Walmart and Sam's Club to require real-time, end-to-end food traceability with blockchain. https://corporate.walmart.com/newsroom/2018/09/24/in-wake-of-romaine-e-coli-scare-walmart-deploys-blockchain-to-track-leafy-greens, 2018.

[43] G. Zyskind, O. Nathan, and A. Pentland. Enigma: Decentralized computation platform with guaranteed privacy. arXiv:1506.03471, 2015.